

The background is a dark navy blue. In the top-left corner, there are two overlapping geometric shapes: a blue parallelogram and a light green parallelogram. In the bottom-left, there is a circular inset showing a detailed, grayscale image of a printed circuit board (PCB) with various electronic components. In the top-right corner, there is a faint, grayscale image of a complex circuit board layout with many traces.

# Pataverse Case Study

A Django-based augmented reality project



# Overview

This code was extracted from a personal project called Pataverse, a Django-based augmented reality platform designed to allow users to create and share location-based AR content without requiring technical knowledge, specialist software or their own AR project setup. This represents a work in progress.

Users can create text-based messages displayed on template-based virtual signs, or associate locations with their own images and 3D content. Each piece of content is positioned geographically, allowing it to appear as part of the physical environment when viewed through the AR interface.

Content can be published to a public channel, making it discoverable by other users, or to a private channel, where it is only visible to the creator's friends. Users can access the AR viewer from a mobile device and use their camera to discover virtual objects positioned around their physical surroundings.



# The Problem

**Creating augmented reality experiences normally requires technical knowledge, development tools and a considerable amount of setup.**

- AR development has a relatively high technical barrier
- Creating location-based content generally requires a custom application
- Users can't easily create and publish their own AR experiences
- Existing AR experiences are often isolated, rather than forming a shared spatial layer

How might we make location-based AR content as easy to create as publishing a post?



# The Concept

**Create → Place → Discover → Experience**

**Create** – Users create digital content, from simple messages and artwork to images and 3D objects.

**Place** – Content is anchored to a real-world location, giving it a meaningful position in physical space.

**Discover** – Other users explore their surroundings through the AR viewer, discovering content positioned around them.

**Experience** – Digital content becomes part of the environment, allowing users to interact with places in new ways and experience locations through stories, art, history and shared experiences.



# Core Features

- **Location-based AR content / Implemented**
  - Geotagged objects presented to users based on their phone's GPS location, updating in real-time as they move around
- **Template-based text signs / Implemented**
  - Pre-made templates that determine the shape, thickness, size, etc of the signs used to display text
- **User-provided images / Partially Implemented**
- **User-provided 3D content / Partially Implemented**
- **Public channels / Implemented**
  - Seasonal channels, like spooky Halloween walks
  - Historical trails using old photos of city landmarks
  - Art exhibitions created by artists to show off their works in virtual galleries
- **Private/friends-only channels / Partially Implemented**
- **Mobile AR viewer / Implemented**
- **User accounts and content ownership / Implemented**



# Technical Architecture

**Browser / Mobile** – Users access the platform through a mobile browser or Progressive Web App, without requiring a dedicated native application.



**AR.js / A-Frame** – Provides browser-based augmented reality, rendering virtual content within the user's physical surroundings.



**Django application** – Handles authentication, content management, permissions, business logic and the core application experience.



**PostgreSQL** – Stores application data, user accounts, artwork metadata and relationships between users and content.



**Geospatial content / user data** – Connects digital content to physical locations while managing associated user and artwork information.



# Geospatial Architecture/01

Location is a core part of Pataverse rather than simply metadata attached to an artwork. Every piece of content is associated with a real-world position, which determines where and when it can be discovered.

## **Coordinate capture**

The user's device provides latitude and longitude when creating or interacting with location-based content.

## **Location storage**

Coordinates are stored alongside each artwork, allowing content to be associated with a specific physical location.

## **Proximity detection**

The application compares the user's location with nearby content to determine what should be available to discover.



# Geospatial Architecture/02

## **Spatial validation**

Submitted coordinates are validated server-side, including checks against restricted areas and acceptable geographic boundaries.

## **Geospatial filtering**

Only relevant content is returned to the client, reducing unnecessary data transfer and keeping the AR experience responsive.

## **Physical → Digital → Physical**

A real-world location becomes data, that data determines what digital content is presented, and the user experiences it back in physical space.



# The Interesting Engineering Problems/01

The interesting part was that these problems are interconnected. A decision in one layer affects the behaviour of several others.

## **Geospatial queries → discovery**

→ Artwork stores **latitude** and **longitude**; the backend compares these against the user's current position and returns content within the discovery radius.

## **Discovery → performance**

→ Rather than sending every artwork to the browser, the backend filters the dataset first, limiting the number of AR objects and assets the client must process.

## **Location → safety**

→ Before saving an artwork, the backend checks its coordinates against each active **RestrictedArea** polygon using a point-in-polygon calculation.



# The Interesting Engineering Problems/02

## **Client → security**

→ Coordinates received from the browser are treated as untrusted. Latitude/longitude ranges, ownership and permissions are checked server-side before database changes are permitted.

## **Content → AR**

→ Uploaded images and 3D files pass through file-type and size validation before being approved and made available to the A-Frame scene.

## **AR → usability**

→ Keeping rendering in the browser avoids a native application while allowing the same Django application to handle authentication, creation, permissions and content management.

## **The architectural result**

→ Django decides what is allowed and what is relevant → A-Frame decides how it is experienced.



# Risks and Mitigations/01

## User-generated content and moderation

### Risks

- Illegal/offensive content → moderation, reporting, takedown processes, terms of service.
- Harassment or targeted abuse → someone could deliberately place offensive AR content at another person's home, workplace, memorial, etc.
- Spam/trolling → users could flood popular locations with low-quality or malicious content.

### Mitigations

- Clear terms of use and community guidelines
- Blocking and reporting functionality
- User whitelisting for riskier content creation such as images and videos
- Automated moderation pipeline for text before any gets posted



# Risks and Mitigations/02

## Geolocation and privacy

### Risks

- Precise location data → Could reveal a user's home, workplace, movements or other sensitive locations
- Location history → Long-term storage could allow detailed patterns of an individual's movements to be reconstructed
- Stalking or harassment → Someone could use location-based content to target another person

### Mitigations

- Data minimisation
- Privacy-aware defaults and sensitive-location controls
- Encryption and strict access controls



# Risks and Mitigations/03

## Abuse of the Location System

### Risks

- Mass artwork creation → A malicious user could create thousands of artworks and overwhelm a location or database
- Artificially dense locations → Popular areas could become unusable because of excessive overlapping content
- Location spoofing → Users could submit false coordinates and bypass location-based restrictions

### Mitigations

- Automated abuse detection
- Server-side location validation
- Multiple channels for content, allowing multiple layers in the same area.



# Risks and Mitigations/04

## Scalability and Infrastructure

### Risks

- Large numbers of geospatial objects → Queries could become increasingly slow as the database grows
- Storage growth → User-uploaded artwork could cause rapidly increasing storage requirements
- Expensive operations → Image processing, mapping requests and other resource-intensive features could become costly at scale

### Mitigations

- Efficient geospatial queries
- Use third party file hosting, which is then embedded into AR content
- Time To Live on posts, which unpublishes them once it expires



# Risks and Mitigations/05

## Physical-World Safety

### Risks

- Distracted users → Looking at an AR experience while walking could cause collisions or traffic accidents
- Dangerous locations → Users could be encouraged towards roads, railways, construction sites or other hazards
- Trespassing → Users could enter private property to find an artwork

### Mitigations

- Clear safety warnings
- Discourage AR use while walking or driving
- Geofencing around dangerous or restricted areas



# Security and Privacy/01

## Security & Privacy

Pataverse combines **user accounts, uploaded content and precise geographical data**, so security and privacy had to be considered at the application and data layers.

### Data minimisation / **Implemented**

Store only the location and user data required for the platform to function, with limited retention.

### Location privacy / **Partially Implemented**

Restrict location precision where possible and prevent unnecessary exposure of sensitive user locations.

### Server-side permissions / **Implemented**

Validate ownership and channel membership on every protected request rather than relying on client-side checks.



# Security and Privacy/02

## **Input validation / Implemented**

Validate coordinates, IDs, text and API parameters server-side, treating all client-provided data as untrusted.

## **Third Party File Handling / Implemented**

In the current implementation, user-generated images and 3D models are stored using third party services and then embedded into Pataverse projects, removing the need for multimedia storage, and easing content moderation.

## **Abuse prevention / Partially Implemented**

Apply rate limits, creation quotas and server-side geospatial validation to prevent automated or malicious use.

## **Privacy by design / Partially Implemented**

Separate public content from private/friends-only content and ensure private location-based content cannot be discovered through public queries or APIs.



# Future Developments

## Near Term

- Better content discovery
- Improved AR positioning
- Moderation tooling
- Accessibility improvements
- Better geofencing for restricted areas
- Finish implementing social features

## Medium Term

- More sophisticated 3D content
- Community features
- Collaborative experiences
- Better spatial indexing

## Long Term

- Persistent shared AR spaces
- Location-based events
- Public art / cultural applications
- Educational and community uses